Mediator

Mediator · Chat room without user-to-user coupling · Build with Waqar

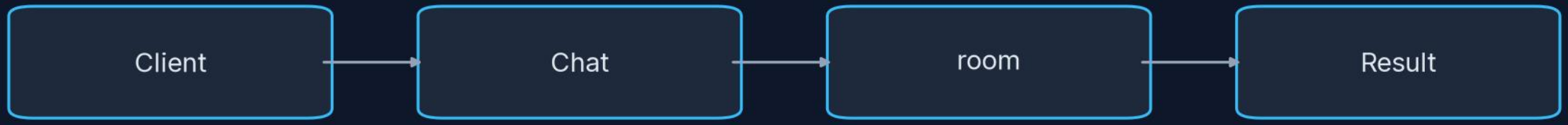
Today's Hook

- Problem: Chat room without user-to-user coupling
- Pattern: Mediator — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Chat room without user-to-user coupling
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Mediator: Chat room without user-to-user coupling

Real-World Scenario

- Running example: Chat room without user-to-user coupling
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.gof.behavioral.mediator.problem;

import java.util.*;

public final class NaiveChat {
  public static void dm(String from, String to, String msg){ System.out.println(from+"->"+to+":"+msg); }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```
package com.buildwithwaqar.patterns.gof.behavioral.mediator.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.function.*;

final class ChatRoom {
  private final Map<String, Consumer<String>> users = new HashMap<>();
  private final List<String> log = new ArrayList<>();
  void join(String name, Consumer<String> inbox){ users.put(name, inbox); }
  void send(String from, String to, String msg){
    var inbox = users.get(to);
    if (inbox==null) throw new IllegalArgumentException("missing:"+to);
    String line = from+"->"+to+":"+msg; log.add(line); inbox.accept(line);
  }
  List<String> log(){ return List.copyOf(log); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.behavioral.mediator.problem;
import java.util.*; import java.util.concurrent.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Mediator: chat room ===");
    System.out.println("--- naive ---"); NaiveChat.dm("Ada","Bob","hi");
System.out.println("--- mediator ---");
var room=new ChatRoom(); List<String> bob=new ArrayList<>();
room.join("Ada", m->{}); room.join("Bob", bob::add);
room.send("Ada","Bob","hi");
System.out.println("bobInbox="+bob); System.out.println("log="+room.log());
try { room.send("Ada","Cara","x"); } catch (IllegalArgumentException e){ System.out.println("caught="+e.getMessage()); }
  }
}
```

Walkthrough Output

```
=== Mediator: chat room ===
--- naive ---
Ada->Bob:hi
--- mediator ---
bobInbox=[Ada->Bob:hi]
log=[Ada->Bob:hi]
caught=missing:Cara
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Mediator.

Related Patterns

- Contrast Mediator with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Mediator → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.